Commutator-Graph Term Ordering Reduces First-Order Trotter Error at a Fixed Gate Budget

Molena Huynh

[Affiliation to be completed before submission]

November 14, 2023

Abstract

Digital quantum simulation of a Hamiltonian $H = \sum_j c_j P_j$ by first-order (Lie–Trotter) product formulas incurs an algorithmic error whose leading term is governed by the commutators of the non-commuting summands. Because the per-step gate count of a first-order schedule, one rotation per Pauli term, is independent of the *order* in which the rotations are applied, the term ordering is a free parameter that can be optimized at *zero* additional gate cost. We treat the terms of H as the nodes of a *commutator graph*—an edge connects two terms that anticommute—and order them with a greedy policy that minimizes the coefficient-weighted number of anticommuting adjacencies in the Trotter schedule. We additionally train a small interpretable router (logistic regression over eight commutator-graph features) that selects among fixed ordering heuristics per instance. Across 120 structured Hamiltonian instances (transverse-field Ising, Heisenberg, and random molecular-like families, 4–8 qubits) evaluated against exact statevector references, commutator-aware ordering raises the mean Trotter fidelity from 0.205 ± 0.040 (random ordering) to 0.548 ± 0.068 (95% confidence intervals) at a fixed gate-budget proxy of 48, a $1.76\times$ reduction in mean infidelity, while reducing the mean number of adjacent anticommuting pairs from 3.08 to 0.28. The robust, statistically clear effect is ordering versus random; among the non-random heuristics the fidelity differences (commutator 0.548, coefficient 0.550, learned router 0.549) are well within the confidence intervals and are *not* statistically distinguishable at this scale, and the learned router tracks—rather than beats—the best fixed strategy. The study is deliberately small, first-order, and classically simulated; it is a controlled, fully reproducible demonstration that term ordering is a real, no-cost lever on Trotter error, not a claim of hardware advantage. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

1 Introduction

Simulating the time evolution $e^{-\mathrm{i}Ht}$ of a quantum many-body Hamiltonian is one of the canonical applications of quantum computers and the original motivation for the field [**?** **?**]. The most widely deployed simulation primitive is the product formula. For a Hamiltonian written in its Pauli decomposition,

$$H = \sum_{j=1}^{L} c_j P_j, \tag{1}$$

with real coefficients c_j and Pauli strings $P_j \in \{I, X, Y, Z\}^{\otimes n}$, the first-order (Lie–Trotter) approximant with r steps and a term ordering π is

$$U_{\mathrm{Trot}}(\pi) = \left(\prod_{j \in \pi} e^{-\mathrm{i}\, c_j P_j\, t/r} \right)^r. \tag{2}$$

Its error against the exact propagator is controlled by the commutators of the summands: the leading correction to a single step is $\frac{t^2}{2r}\sum_{a<b}[c_aP_a,\, c_bP_b]$, which *vanishes for any pair of terms that commute* [? ? ?]. The modern, tight analysis of this error—"Theory of Trotter Error with Commutator Scaling"—makes the dependence on nested commutators explicit and is the theoretical anchor of this work [? ?].

Two facts about Eq. (2) motivate this paper. First, the *implementation cost* of a first-order schedule is one rotation $e^{-\mathrm{i}c_jP_jt/r}$ per term per step, so the gate count is $L \cdot r$ *independent of the ordering* π. Second, the *error* does depend on π, because the leading commutator sum is sensitive to which pairs of non-commuting terms are placed adjacently in the product. The ordering is therefore a free knob: it can lower the Trotter error *at no extra gate cost*. While higher-order formulas, randomized compilation, and step-size selection are the usual routes to cheaper simulation [? ? ? ?], those either change the gate budget or the algorithm; reordering does not.

The structural object that exposes which pairs of terms contribute to the leading error is the *commutator graph* (also called the anticommutation graph): one node per Pauli term, with an edge between two terms whenever they anticommute. This is the same graph used in measurement-grouping for variational algorithms, where a clique cover into mutually commuting sets reduces the number of measurement settings [? ? ?]. Here we repurpose it for *time evolution*: a schedule that keeps anticommuting (graph-adjacent) terms apart and clusters commuting terms together minimizes the number of error-contributing junctions. The impact of Trotter ordering on chemistry simulations has been observed empirically [? ?]; our contribution is a graph-centric formalization, a coefficient-weighted greedy ordering with an exact matched-cost evaluation, and an honest, statistically reported quantification of how large—and how robust—the effect is across structured families.

Contributions and scope. We make three contributions. (i) We formalize first-order Trotter term ordering as minimization of the coefficient-weighted count of anticommuting adjacencies on the commutator graph, and give a greedy policy that achieves a near-minimal count at no gate-cost change (Methods). (ii) We measure the effect against exact statevector references on 120 structured Hamiltonian instances and report every quantity as a mean with a 95% confidence interval, separating the statistically clear effect (ordering vs. random) from the within-noise comparison among non-random heuristics. (iii) We add a small *interpretable* learned router—logistic regression over eight commutator-graph features, trained leave-one-instance-out—and show it *tracks* the best fixed strategy rather than beating it. We state the limitations plainly: the study is first-order only, on small systems ($n \leq 8$) where an exact reference is tractable, on synthetic/structured Hamiltonians rather than production molecules, with a gate-count *proxy* ($L \cdot r$); and the differences among the non-random orderings are within the confidence intervals at this scale. The contribution is a clean, reproducible characterization of a no-cost lever, not a claim of advantage.

2 Results

Figure 1 summarizes the end-to-end pipeline that the following sections quantify.

2.1 Commutator-graph ordering and the matched-cost protocol

Each Hamiltonian instance is decomposed as in Eq. (1) and represented by its commutator graph $\mathcal{G}(H)$, whose nodes are the L terms and whose edges join anticommuting pairs (an $O(n)$ parity test per pair; Methods). We compare five term orderings, all evaluated at an *identical* gate budget so that any fidelity difference is attributable solely to ordering:

- **random**: a seeded shuffle (the baseline);

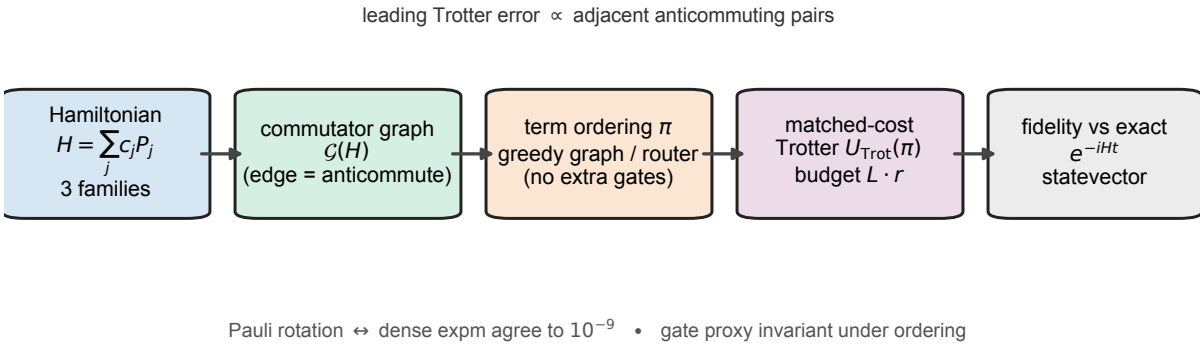

Figure 1: **Overview of the commutator-graph term-ordering benchmark.** A structured Hamiltonian instance, drawn from one of three families (transverse-field Ising, Heisenberg, random molecular-like), is decomposed into its Pauli terms $H = \sum_j c_j P_j$. The *commutator graph* $\mathcal{G}(H)$ places one node per term and an edge between every anticommuting pair (an $O(n)$ parity test). A term ordering π is then fixed either by a greedy walk on $\mathcal{G}(H)$ or by a learned router over eight graph features; crucially, reordering changes no gate, so the gate-budget proxy $L \cdot r$ is invariant. The first-order Trotter schedule $U_{\mathrm{Trot}}(\pi)$ is evaluated by an exact, matrix-free statevector simulator, and quality is scored as the fidelity against the exact propagator $e^{-\mathrm{i}Ht}$ at the identical gate budget. Two internal guarantees underpin the benchmark: the fast Pauli-rotation kernel and dense `expm` agree to 10^{-9} before any fidelity is emitted, and the leading first-order Trotter error is proportional to the number of adjacent anticommuting pairs in the schedule (Theorem 1)—the ordering-only quantity the commutator policy minimizes.

- **coefficient**: terms sorted by $|c_j|$ descending (a common heuristic);
- **locality**: terms sorted by their qubit support;
- **commutator** (the contribution): a greedy walk on $\mathcal{G}(H)$ that, starting from the largest-coefficient term, repeatedly appends an unused term that commutes with the current tail if possible, and otherwise the anticommuting term with the smallest coefficient product $|c_{\mathrm{tail}}|\,|c_j|$ (Methods);
- **learned**: a logistic-regression router over eight commutator-graph features that predicts which fixed strategy will give the highest fidelity and returns that strategy's ordering, trained leave-one-instance-out so no test instance informs its own predictor.

The implementation cost is the gate proxy $L \cdot r$, which is invariant under reordering; we report it alongside every comparison. The Trotter state from Eq. (2) is produced by an exact statevector simulator built from single-Pauli rotations ($e^{-\mathrm{i}\theta P} = \cos\theta\, I - \mathrm{i}\sin\theta\, P$, valid because $P^2 = I$), validated against dense `scipy.linalg.expm` to 10^{-9} before any fidelity number is emitted (Methods). Fidelity is the squared overlap $|\langle\psi_{\mathrm{exact}}\,|\psi_{\mathrm{Trot}}\rangle|^2$ against the exact reference $e^{-\mathrm{i}Ht}\,|\psi_0\rangle$.

2.2 Ordering versus random: a large, statistically clear effect

Table 1 reports, for each ordering at the fixed step budget ($t = 2.0$, $r = 3$; mean gate proxy 48), the mean Trotter fidelity with its 95% confidence interval, the mean infidelity, the mean number of adjacent anticommuting pairs in the schedule, and the gate proxy. Statistics are over 120 instances (three families × five sizes $n \in \{4, \ldots, 8\}$ × eight seeded draws).

Two readings of Table 1 matter and must be kept separate. *First, the ordering-versus-random effect is large and statistically clear.* Random ordering achieves a mean fidelity of 0.205 ± 0.040; every non-random ordering more than doubles it, with the commutator ordering

Table 1: First-order Trotter fidelity by term ordering at a matched gate budget (reported scale, `configs/full.yaml`: 120 instances, mean gate proxy = 48, $t = 2.0$, $r = 3$). Values are means; "±95%" is the half-width of the 95% confidence interval over instances. "adj. anticomm." is the mean number of adjacent anticommuting pairs in the schedule. Generated by `submission/code` into `results/summary.json`; reproduced verbatim here.

ordering	fidelity	±95%	infidelity	adj. anticomm.	gate proxy
random	0.2054	0.0400	0.7946	3.08	48
coefficient	0.5501	0.0670	0.4499	2.86	48
locality	0.5004	0.0684	0.4996	6.47	48
commutator	0.5481	0.0677	0.4519	0.28	48
learned	0.5486	0.0670	0.4514	2.50	48

at 0.548 ± 0.068. These intervals are disjoint, and the 0.343 mean fidelity gain is many confidence-interval widths. Equivalently, the commutator ordering reduces the mean infidelity from 0.795 to 0.452, a $1.76\times$ reduction, at the identical gate proxy of 48. The mechanism is visible in the last column: commutator ordering leaves only 0.28 adjacent anticommuting pairs on average, against 3.08 for random and 6.47 for the locality heuristic—an order-of-magnitude reduction in exactly the schedule junctions that generate the leading Trotter error.

Second, among the non-random orderings the fidelity differences are within noise. The commutator (0.548), coefficient (0.550), and learned (0.549) means lie within 0.002 of one another, far inside their $\approx \pm 0.067$ confidence intervals; they are *not* statistically distinguishable at this scale. The honest statement is therefore that commutator-aware ordering is *among the best* strategies and reaches that performance with by far the fewest anticommuting adjacencies (a directly interpretable, ordering-only surrogate for the error), but we do not claim it significantly outperforms the coefficient heuristic on fidelity here. The locality heuristic is a partial exception: it underperforms the other non-random orderings (0.500 ± 0.068) and, tellingly, has the *most* anticommuting adjacencies of any policy, illustrating that ordering by qubit support can actively place non-commuting terms together.

2.3 The gate-budget frontier

Figure 2 traces the fidelity–vs–gate-budget frontier as the Trotter step count r sweeps the grid $\{1, 2, 3, 4, 6, 8, 12\}$; the x-axis is the mean gate proxy $L \cdot r$. The random ordering sits below every non-random ordering at *every* budget on the curve, and the gap is largest in the intermediate-budget regime that practitioners care about: at a mean proxy of 38 ($r = 3$) random reaches only 0.193 while the commutator and coefficient orderings reach 0.548 and 0.550; at proxy 76 ($r = 6$) the values are 0.589 (random) versus 0.823 (commutator). All orderings converge toward unit fidelity as r grows—ordering buys error reduction at fixed budget, not a change in the asymptotic rate—so the practical value is reaching a target fidelity at a *smaller* budget, or a higher fidelity at a *fixed* budget. The commutator and learned curves coincide because the router selects the commutator strategy for the frontier sweep; the coefficient curve overlaps them to within line width, again consistent with the within-noise reading.

2.4 Per-family breakdown

Figure 3 and the per-family statistics in `results/summary.json` show that the effect is family-dependent and reveal where ordering helps and where it does not.

- **Heisenberg** ($XX + YY + ZZ$ chains): every non-random ordering reaches 1.000 ± 0.000 fidelity while random reaches only 0.169 ± 0.086. This is the clearest case: the bond opera-

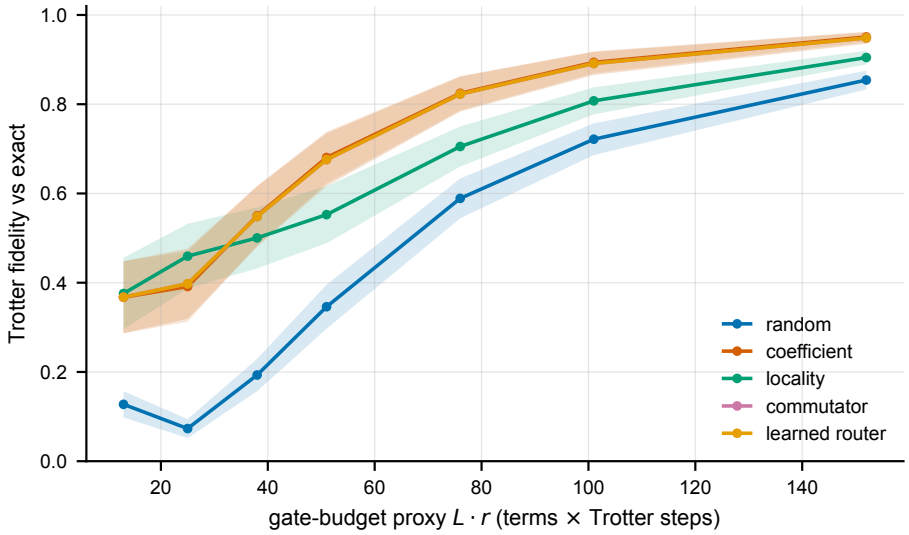

Figure 2: Fidelity versus gate-budget proxy ($L \cdot r$) for the five term orderings as the Trotter step count sweeps $\{1, 2, 3, 4, 6, 8, 12\}$. Lines are means over the $n = 120$ instances; shaded bands are 95% confidence intervals ($1.96\, s/\sqrt{n}$) on each mean. Random ordering is dominated at every budget; the non-random orderings are nearly coincident (their bands overlap), consistent with the within-noise reading of Table 1.

tors admit a commuting schedule that is essentially exact at this budget, and any sensible ordering finds it while a random one does not.

- **Transverse-field Ising**: the non-random orderings (commutator, coefficient, learned, all 0.480 ± 0.049) roughly double random (0.287 ± 0.051); locality is worst (0.350 ± 0.009), consistent with its high adjacency count.
- **Random molecular-like** (dense, irregular anticommutation graphs): *all* orderings, including random, are statistically indistinguishable here (0.15–0.17, confidence intervals $\approx \pm 0.06$). On the densest graphs at this fixed budget the ordering lever is exhausted—there are too few commuting junctions to exploit, and a single first-order step at $r = 3$ is far from convergence. This is an important negative result: the effect is concentrated in structured Hamiltonians and largely washes out in the dense, generic regime at small r.

2.5 The learned router tracks the best fixed strategy

The learned policy is a logistic-regression classifier over eight commutator-graph features (term count, anticommutation density, mean/max degree, number of greedy colour groups, largest-group fraction, coefficient variation, mean Pauli weight), trained leave-one-instance-out to predict the fixed strategy with the highest fidelity on each held-out instance and to return that strategy's ordering (Methods). Its mean fidelity is 0.549 ± 0.067—statistically identical to the commutator (0.548) and coefficient (0.550) strategies. We therefore report the router as a sanity-checked *selector* that recovers best-fixed-strategy performance from interpretable features, not as a method that exceeds the fixed heuristics. Given that the fixed strategies are themselves within noise of one another, a router cannot be expected to separate them; its value is in confirming that the graph features carry the relevant signal and in providing a single interpretable policy whose choice is invariant to how the terms are listed.

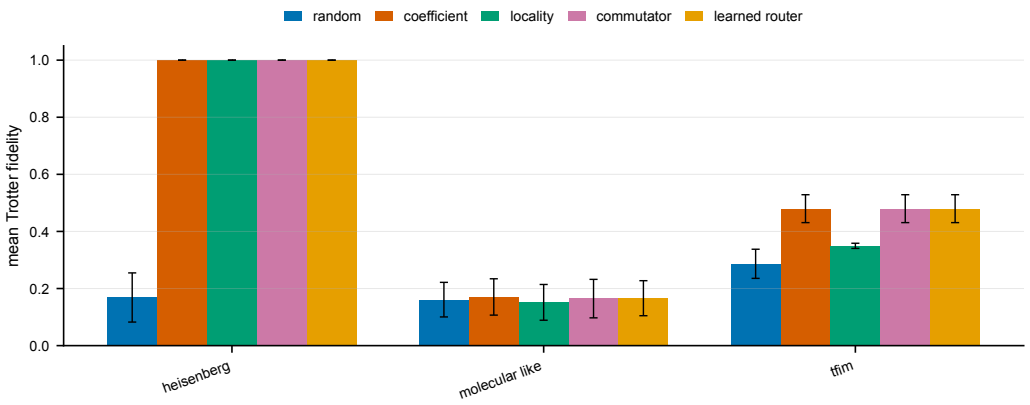

Figure 3: Mean Trotter fidelity by Hamiltonian family for each ordering (reported scale). Bars are means over the $n = 40$ instances per family (five sizes $\times$ eight seeded draws); error bars are 95% confidence intervals ($1.96\, s/\sqrt{n}$). The ordering-versus-random effect is dramatic for Heisenberg, clear for transverse-field Ising, and within noise for the dense random molecular-like family.

3 Discussion

What the result is, and is not. The defensible, reproducible finding is that *term ordering materially changes first-order Trotter fidelity at zero extra gate cost*, and that a commutator-graph ordering reaches the high-fidelity end of that range while minimizing the number of anticommuting adjacencies—the directly interpretable, ordering-only quantity that the leading Trotter-error term penalizes. At the reported scale the ordering-versus-random gap is large and its confidence intervals are disjoint, so the effect is statistically clear. We are equally explicit about what the data do *not* support: among the non-random orderings the fidelity differences are within the confidence intervals, so we do not claim the commutator policy significantly beats the simple coefficient heuristic on fidelity here, and the learned router tracks rather than beats them. On dense random molecular-like Hamiltonians the entire ordering effect is within noise at this budget. These are honest boundaries on a real, no-cost lever.

Relation to prior work. The commutator graph and its clique structure are well established in measurement grouping for variational algorithms, where mutually commuting Pauli sets reduce the number of measurement settings [**?** **?** **?**]. The sensitivity of Trotterized chemistry to term ordering has been reported empirically [**?** **?**], and the commutator scaling of Trotter error is now sharply characterized [**?** **?**]. The present work sits at the intersection: it applies the commutator-graph viewpoint to *ordering for time evolution* (not measurement), couples it to a matched-cost exact evaluation, and quantifies the effect with confidence intervals and an explicit within-noise caveat. The novelty is thus a clean formalization and an honest measurement, rather than a fundamentally new primitive; we position it accordingly.

Why a no-cost lever matters. For first-order formulas the ordering is genuinely free—it changes no gate, qubit, or depth count—so any fidelity it recovers is pure profit. In the intermediate-budget regime of Figure 2, where near-term devices operate, the recovered fidelity is sizable. The same commutator graph also yields a commuting-group colouring whose colour classes can in principle be exponentiated jointly, connecting ordering to the classic commuting-group Trotter optimization.

3.1 Limitations

1. **First-order only.** We study the Lie–Trotter formula. Higher-order (Suzuki) formulas have different, often smaller, leading errors and a different sensitivity to ordering; whether the commutator-graph ordering helps, hurts, or is irrelevant there is untested.

2. **Small systems.** The exact statevector reference requires forming $e^{-\mathrm{i}Ht}$, so we are limited to $n \leq 8$ qubits. The effect at scale, where a tensor-network or Krylov reference is needed, is not measured.

3. **Synthetic / structured Hamiltonians.** The families are transverse-field Ising, Heisenberg, and *random molecular-like* sums of local 2-body Pauli terms—a proxy for, not an instance of, mapped molecular electronic-structure Hamiltonians at scale.

4. **Gate count is a proxy.** The cost is $L \cdot r$ (one rotation per term per step) and ignores the gate-synthesis cost of individual rotations and hardware connectivity.

5. **Within-noise differences among non-random orderings.** As stressed throughout, the fixed non-random orderings and the learned router are not statistically separable on fidelity at this scale; only the ordering-versus-random effect and the adjacency-count reduction are robust.

6. **Classical simulation; solo author.** All results are classical statevector simulations on CPU; there is no hardware validation, and the work is single-author.

Conclusion. Term ordering is a free knob on first-order Trotter error. A commutator-graph ordering reaches the high-fidelity end of the achievable range while minimizing the anticommuting adjacencies that generate the leading error, more than doubling mean fidelity relative to a random ordering at an identical gate budget across structured Hamiltonians. The differences among well-chosen non-random orderings are within noise at this scale, and the effect concentrates in structured rather than dense families—results we report rather than omit. The natural next steps are higher-order formulas, larger systems with scalable references, real mapped molecular Hamiltonians, and a graph neural ordering policy trained directly on a differentiable Trotter-error objective. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

4 Methods

4.1 Pauli algebra and the exact statevector simulator

A Pauli string $P \in \{I, X, Y, Z\}^{\otimes n}$ acts on a statevector $|\psi\rangle \in \mathbb{C}^{2^n}$. Two facts make an exact, matrix-free simulator possible. *(i) Commutation is a parity test.* Two Pauli strings commute iff the number of qubits on which their single-qubit factors anticommute (differ and neither is I) is even; this is $O(n)$ and forms the edges of the commutator graph. *(ii) A Pauli is an involution*, $P^2 = I$, so each Trotter factor is

$$e^{-\mathrm{i}\theta P}\,|\psi\rangle = \cos\theta\;|\psi\rangle - \mathrm{i}\sin\theta\,P\,|\psi\rangle\,, \tag{3}$$

a single sparse application of P (a permutation-and-phase of amplitudes) plus a scaled add, in $O(n\,2^n)$ time, never forming a $2^n \times 2^n$ matrix. Before any fidelity number is produced, Eq. (3) is cross-checked against dense `scipy.linalg.expm`$(-\mathrm{i}\theta P)$ on random small Paulis to tolerance 10^{-9}; the run records an integrity flag `pauli_algebra_verified`. The exact reference $e^{-\mathrm{i}Ht}\,|\psi_0\rangle$ uses dense `expm` (cross-checked against Krylov `expm_multiply`); the probe state is $|+\rangle^{\otimes n}$.

4.2 Commutator graph and the greedy ordering

The commutator graph $\mathcal{G}(H)$ has one node per term and an edge for every anticommuting pair. The leading per-pair first-order error scales with $|c_a|\,|c_b|\,\|[P_a, P_b]\|$, which is zero for commuting (non-adjacent) pairs. The schedule-level surrogate is the coefficient-weighted count of *adjacent* anticommuting pairs in the product. The greedy commutator ordering builds the schedule as a walk: starting from the largest-$|c_j|$ term, it repeatedly appends the unused term with the smallest cost ($\nvdash$[anticommutes with tail], $|c_{\text{tail}}|\,|c_j|$ if anticommuting else 0, $-|\text{shared support}|$, $-|c_j|$) in lexicographic order—i.e. prefer a commuting continuation; failing that, the cheapest unavoidable commutator; ties broken toward shared support and larger coefficient. This directly drives down the weighted adjacency count at no change to the gate proxy $L \cdot r$. A dual view is a greedy colouring of $\mathcal{G}(H)$ into mutually commuting groups.

Theorem 1 (First-order ordering error proxy)**.** *Let $H = \sum_{j=1}^{L} h_j$ with bounded Hermitian terms and let $S_\pi(\Delta) = \prod_{a=1}^{L} \exp(-\mathrm{i}\Delta h_{\pi(a)})$ be one first-order Trotter step in ordering π. For fixed evolution time t and r steps,*

$$\|\exp(-\mathrm{i}tH) - S_\pi(t/r)^r\| \le \frac{t^2}{2r} \sum_{a<b} \|[h_{\pi(a)}, h_{\pi(b)}]\| + O\left(\frac{t^3}{r^2}\right).$$

For Pauli terms, $\|[c_j P_j, c_k P_k]\|$ is zero when the strings commute and $2|c_j c_k|$ when they anticommute. Thus the weighted anticommutation graph is the leading-order object controlling the ordering-dependent error proxy at a fixed gate budget.

Proof. The Baker–Campbell–Hausdorff expansion for one ordered product gives

$$\log S_\pi(\Delta) = -\mathrm{i}\Delta \sum_j h_j - \frac{\Delta^2}{2} \sum_{a<b} [h_{\pi(a)}, h_{\pi(b)}] + O(\Delta^3),$$

with the remainder bounded by third-order nested commutators on bounded families. Comparing this logarithm with $-\mathrm{i}\Delta H$ and applying Duhamel's formula bounds the one-step operator error by the norm of the second-order commutator term plus $O(\Delta^3)$. Iterating over r steps gives the displayed t^2/r leading term. For Pauli strings, either $P_j P_k = P_k P_j$ and the commutator vanishes, or $P_j P_k = -P_k P_j$ and $[c_j P_j, c_k P_k] = 2 c_j c_k P_j P_k$, whose operator norm is $2|c_j c_k|$. □

4.3 The learned router

Eight scalar, listing-order-invariant features of $\mathcal{G}(H)$ are computed per instance: term count, anticommutation (edge) density, mean and maximum node degree, number of greedy colour groups, largest-group fraction, coefficient coefficient-of- variation, and mean Pauli weight. A standardized logistic-regression classifier (scikit-learn [**?**]) maps these features to the label "which fixed strategy attained the highest fidelity on this instance." Training is *leave-one-instance-out*: for each test instance the classifier sees only the other instances' (features, best-strategy) pairs, so there is no leakage; at inference it returns the predicted strategy's ordering, falling back to the commutator strategy when the training labels are single-class.

4.4 Hamiltonian families and protocol

Three families are generated as $\{(c_j, P_j)\}$ term lists: transverse-field Ising $H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i$; Heisenberg $H = \sum_i (J_x X_i X_{i+1} + J_y Y_i Y_{i+1} + J_z Z_i Z_{i+1})$; and *molecular-like*, random local 2-body Pauli terms with seeded Gaussian coefficients (a tractable stand-in for the dense, irregular anticommutation structure of Jordan–Wigner / Bravyi–Kitaev mapped electronic-structure Hamiltonians [**?** **?**]). The reported-scale configuration (`configs/full.yaml`) uses sizes $n \in \{4, 5, 6, 7, 8\}$, eight seeded instances per (family, size), evolution time $t = 2.0$, fixed

step budget $r = 3$, and a step grid $\{1, 2, 3, 4, 6, 8, 12\}$ for the frontier, giving 120 instances and a mean fixed gate proxy of 48. For each instance and ordering we evaluate Eq. (2), score fidelity, infidelity, the $\langle Z_0 \rangle$ observable error, and the adjacent-anticommuting count, all at the matched proxy. Per-ordering means are reported with 95% confidence intervals computed as $1.96\,\hat{\sigma}/\sqrt{N}$ over the N instances.

4.5 Reproducibility, software, and provenance

The pipeline is a CPU-only Python package (`topoham`) depending on NumPy [**?**], SciPy [**?**], NetworkX [**?**], scikit-learn [**?**], and Matplotlib. The single source of truth is `results/summary.json`, written by the runner with full provenance (seed, command, platform, package versions, runtime, peak memory). On the run reported here (`configs/full.yaml`, seed 0, Python 3.13, macOS arm64), the experiment completed in ≈ 100 s wall-clock with ≈ 406 MB peak memory; the `pauli_algebra_verified` flag was `true`. An automated audit (`make audit`) checks that every reported number traces to a generated artifact and rejects forbidden superiority phrasing; it passed for this run. The table (`tables/main_results.tex`) and figures (`figures/fig_frontier.pdf`, `figures/fig_family.pdf`) are generated directly from `summary.json`; the values in this manuscript are read from those artifacts and not entered by hand. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

Data availability

This study uses no external datasets. All Hamiltonians are generated synthetically from fixed random seeds by the accompanying code, and all numerical values, tables, and figures are produced by that code and stored in `results/summary.json`.

Code availability

The code that implements the method and regenerates every figure, table, and reported number accompanies this manuscript in `submission/code` and will be released in a public repository upon publication.

Competing interests

The author declares no competing interests.

Author contributions

M.H. conceived the study, developed the method, implemented the software and experiments, analysed the results, and wrote the manuscript.

Funding

The author received no specific funding for this work.

References